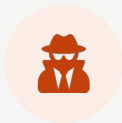


litellm-rs

A self-hosted AI gateway in Rust — keys, budgets, limits, and attribution for every team that touches a model.

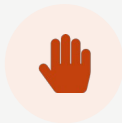
The invoice nobody can explain

What happens when teams share raw provider API keys



No attribution

One shared key, one lump-sum invoice. Nobody can say which team, app, or script spent what.



No limits

A misconfigured batch job or a runaway loop can burn through the monthly budget before anyone notices.

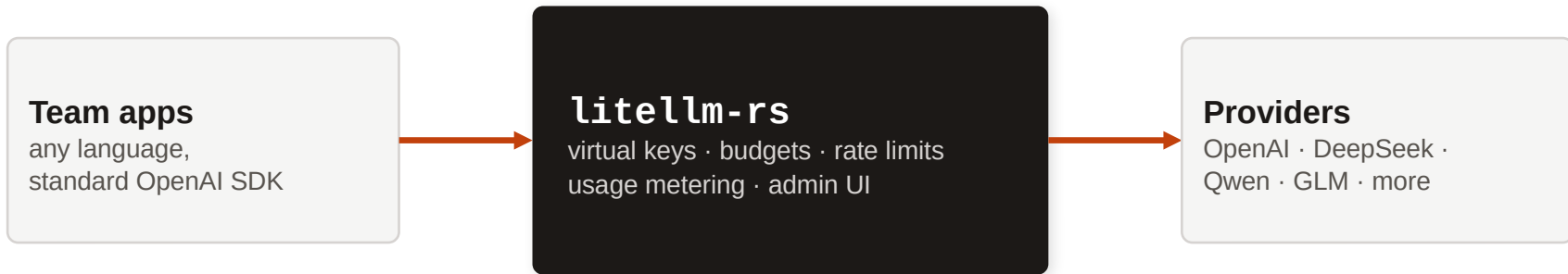


Shared keys, shared risk

The key lives in a dozen .env files. One leak means rotating everything, everywhere, at once.

What is built

One gateway between every team and every provider — every request keyed, metered, and capped.



Three deliberate decisions



Passthrough, not adapters

Provider-native OpenAI wire format end to end. Zero translation tax; streaming stays untouched.



Single binary, not microservices

Right-sized for the real load. One Rust binary + Postgres — cheap to run, trivial to deploy.



Security done properly

argon2-hashed virtual keys, AES-256-GCM provider keys at rest, audit log. Production mindset, not demo mindset.

Live demo

Four beats — from a fresh key to the system saying no.

1



Issue a virtual key

Admin dashboard → create a scoped key with a budget and rate limit. No provider secret ever leaves the server.

2



Make a request

Standard OpenAI SDK call with the new key. Streaming response, live.

3



Watch the meter

Tokens and spend appear against that key in the dashboard — attribution, per request.

4



Hit the cap

Exceed the budget → clean, informative rejection. The system enforcing limits is the whole point.

Beat 4 is the money shot: a system that says no, gracefully, is the value proposition made visible.

Where this sits in the full platform

Mapped against the reference architecture in Designing AI Systems (Manning, 2026) — I built the foundation layer first, deliberately.

Key management & multi-tenant security	SHIPPED	Deeper than the reference — the layer most tools do shallowly
Cost tracking & per-key budget caps	SHIPPED	Per-request metering with hard enforcement
Rate limiting (requests + tokens/min)	SHIPPED	Single-node today; multi-node on the roadmap
Streaming & embeddings proxy	SHIPPED	Usage captured mid-stream
Guardrails & policy enforcement	PLANNED	PII screening · output checks · audit trail
Provider failover & cost-aware routing	PLANNED	Pricing table already in place — the cheap next step
Tracing, evals & experimentation	PLANNED	Compare prompts & models with data, not opinion
Built-in playground (dashboard)	PLANNED	Mint a test key & try it — without leaving the admin UI
Sessions · RAG · tools · workflows	FUTURE	Known, scoped out — not accidental gaps

Where I'd take it next

Sequenced by value, not by novelty.

First



Guardrails & policy enforcement

PII screening, injection defense, and output checks on every request — with audit-ready verdicts. Table stakes for regulated data.

Second



Automatic failover & smart routing

Provider outage? Apps never see it. Requests route by cost and capability — the pricing table this needs already exists.

Third



Tracing, evals & experimentation

Every request traced end to end and scored for quality — then A/B experiments compare prompts and models with data, not opinion.

Fourth



A shared internal platform

Any team onboards in an afternoon: get a key, point the SDK at the gateway, ship.

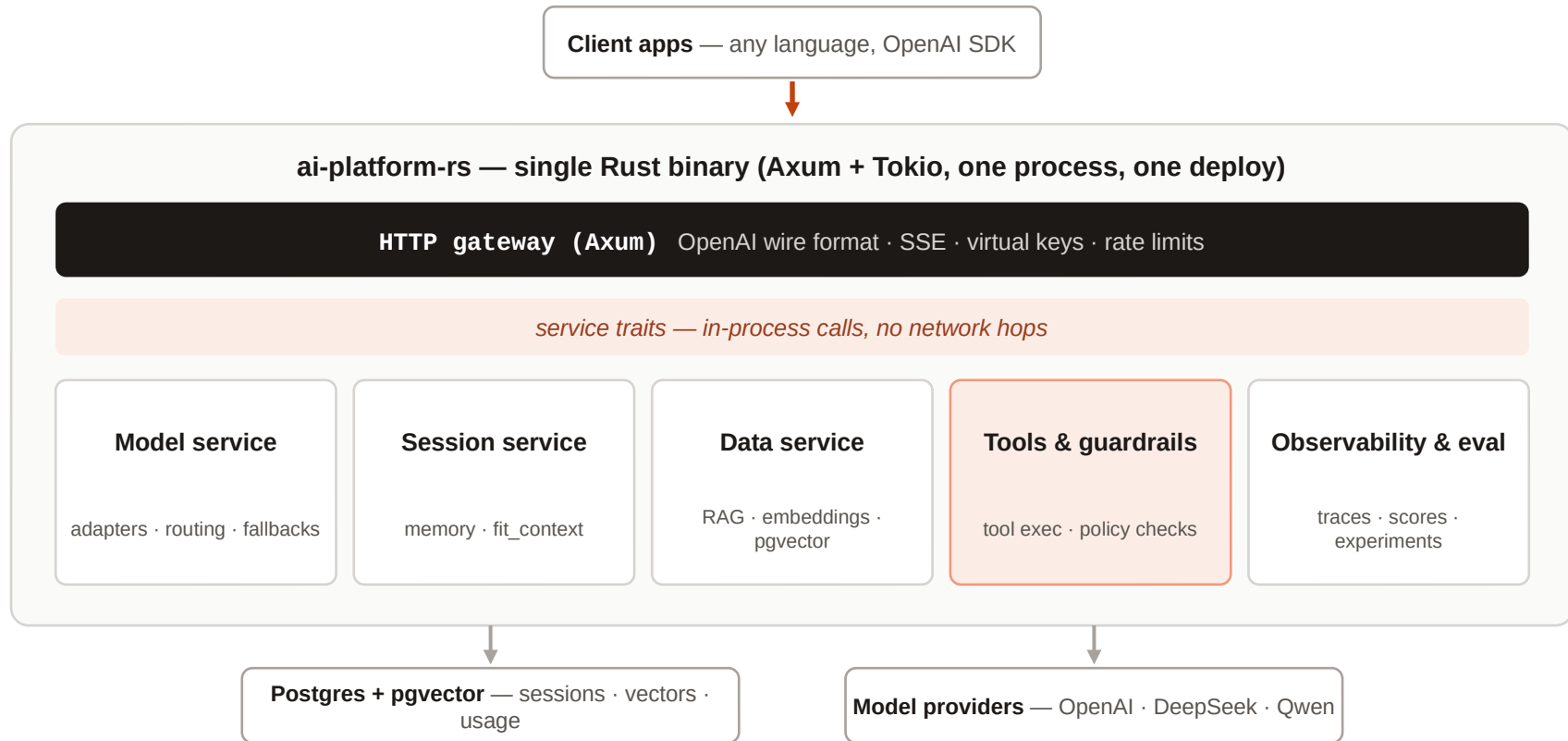
Thank you

One gateway: keys, budgets, limits, attribution — running today.

Siva Krishna Karasala · github.com/sivakarasala/litellm-rs · litellm-rs-43r6l.ondigitalocean.app

The full platform, one Rust binary

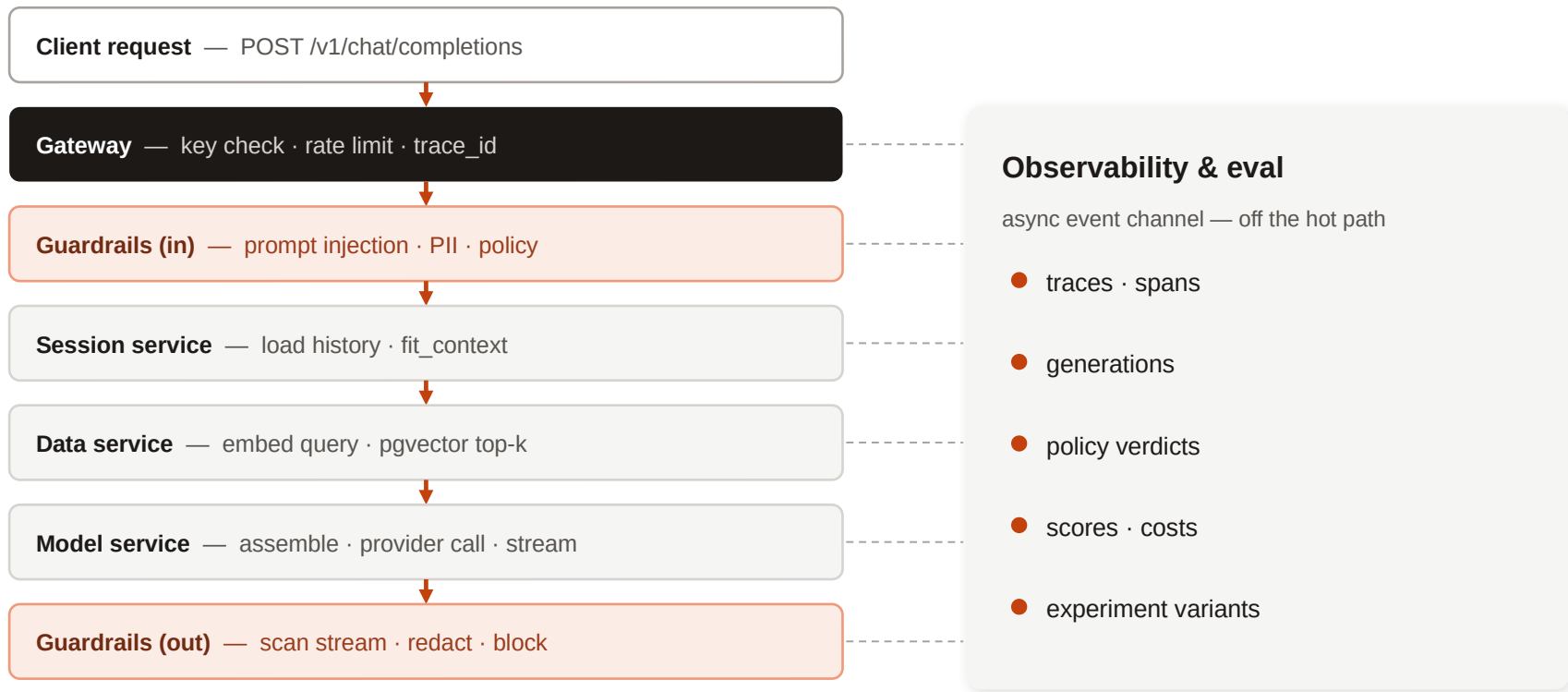
APPENDIX



The book's gRPC service contracts become Rust traits — any module can be extracted to a network service later without changing callers.

One request through the platform

APPENDIX



↺ vetted tokens stream back to the client as SSE

Two guardrail checkpoints — plus a third gating tool calls inside the model step. Every verdict lands in the trace.

Any OpenAI SDK, two lines changed

APPENDIX

Node.js

```
import OpenAI from "openai";

const client = new OpenAI({
  apiKey: "sk-...",
  baseURL: "https://litellm-rs-43r6l.ondigitalocean.app/v1",
});

const stream = await client.chat.completions.create({
  model: "gpt-4o-mini",
  messages: [{ role: "user", content: "Hello" }],
  stream: true,
});

for await (const c of stream)
  process.stdout.write(c.choices[0]?.delta?.content ??
    "");
```

Python (Colab-ready)

```
from openai import OpenAI

client = OpenAI(
    api_key="sk-...",
    base_url="https://litellm-rs-43r6l.ondigitalocean.app/v1",
)

stream = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Hello"}],
    stream=True,
)

for chunk in stream:
    print(chunk.choices[0].delta.content or "", end="")
```

Orange lines are the only change from stock OpenAI usage. Everything else — models, streaming, embeddings, error handling — is the standard SDK, in any language.

Runs anywhere server-side: local scripts, CodeSandbox / Replit (Node), Google Colab (Python).

No code at all: Swagger UI at `/api/swagger-ui` — Authorize with a key, Try it out (use `stream: false`), same origin, no CORS.